Montisci Matteo e Zero Edoardo

Valutazione della complessità tempo dell'algoritmo che calcola il guscio convesso

Algoritmo utilizzato per calcolare la lista contenente il guscio convesso:

```
def crea_guscio(lista_punti):
    lista_guscio=[]
    MIN=lista_punti.pop()
    lista_guscio.append(MIN)
    while len(lista_punti)!=1:
        P=lista_punti.pop()
        if controllo(lista_guscio[-1],P,lista_punti[-1])==True:
            lista_guscio.append(P)
        else:
            if len(lista_guscio)!=1:
                lista_punti.append(lista_guscio.pop())
    MAX=lista_punti.pop()
    lista_guscio.append(MAX)
    return lista_guscio
```

L'algoritmo utilizza la funzione "*controllo*,, che verifica se un punto P è sopra la retta passante per due punti A e B, qui sotto si riporta il codice:

```
def controllo(A,P,B):
    if (P[0]-A[0])*(B[1]-A[1])<(P[1]-A[1])*(B[0]-A[0]):
        return True
```

L'algoritmo utilizza questa funzione, perciò, è essenziale per il calcolo della sua complessità tempo.

Valutazione del Caso Peggiore

SE N=2:

Il ciclo while gira 0 volte perché il minimo e il massimo vanno sempre accettati prima e dopo il ciclo.

Complessità tempo prima del ciclo: Tp(n)=3 (3 ASSEGNAZIONI)

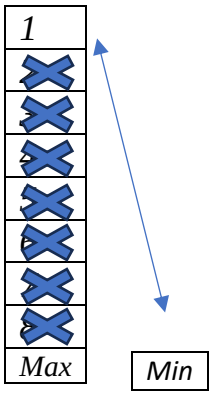
Complessità tempo dopo il ciclo: Td(n)=3 (2 ASSEGNAZIONI E 1 RETURN)

Allora T(N=2)=6 .

SE N>2:

Abbiamo consolidato che il caso in cui l'algoritmo gira più volte è quando accetta il primo punto (non considerando il MIN), poi rifiuta il secondo, riaccetta P(1) (che nel passaggio prima è tornato nella lista L), rifiuta il terzo, riaccetta P(1), e così via. Quindi il '*caso peggiore*' è quando accetta alternativamente i punti, continuando ad accettare P(1) e rifiutando tutti gli altri punti (escluso il MAX e il MIN). La lista finale dei punti contenente il guscio convesso allora conterrà al massimo 3 punti (MAX, MIN e P(1)) e, quindi, né rifiuterà minimo n-3. Guardando l'algoritmo, vedremo che accettare o meno il punto P(1) non crea differenze sul calcolo della complessità tempo.

Esempio con n=10



1 ok	5 no	1 ok o no (non cambia il numero di cicli)
2 no	1 ok	
1 ok	6 no	
3 no	1 ok	
1 ok	7 no	
4 no	1 ok	
1 ok	8 no	

Il ciclo while gira 15 volte

In generale il ciclo gira 2n-5 volte nel peggiore dei casi.

Due peggiori Casi:

1) Accetta n-2 punti e ne rifiuta n-3
2) Accetta n-3 punti e ne rifiuta n-2

Quanto costa accettare un punto?

Accettare un punto ha un costo di 6 iterazioni [1 confronto del while + 1 assegnazione (punto P) + 1 confronto dell'if + 1 confronto della funzione controllo + 1 return + 1 assegnazione (append)]

Quanto costa rifiutare un punto?

Rifiutare un punto costa due iterazioni in più rispetto ad accettarlo, quindi **costerà 8** [in più rispetto all'accettazione di un punti contiene queste iterazioni: 1 confronto (if sulla lunghezza) + 1 acceso a campo (pop) + 1 assegnazione (append) ; ma non contiene l'assegnazione 'lista_guscio.append(P)']

> Quando rifiuta l'ultimo punto però la lunghezza della '*lista_guscio*' è uguale a 1, quindi l'if controlla che la lunghezza della lista sia diversa da 1 e poi non entra nel ciclo, non svolge le ultime due istruzioni elementari (pop e append). In questo caso quindi il costo è ancora **6.**

Quindi i due casi (1 e 2) hanno lo stesso costo: il ciclo while compie n-3 cicli da 8 istruzioni elementari e n-2 cicli da 6 istruzioni elementari.

La complessità tempo dell'algoritmo che calcola il guscio convesso è quindi data dalla somma delle 3 istruzioni elementari prima di entrare nel ciclo while, dalle 3 istruzioni dopo la fine del ciclo while e da tutte le istruzioni che avvengono nel ciclo.

In conclusione si ottiene: **T(n) = 3+3+(n-3)*8+(n-2)*6 = 14n-30** , **Perciò T(n) appartiene a O(n)!**